

ASSIGNMENT- 11.3

Name:MD ARSHAD

HT.No: 2303A51334

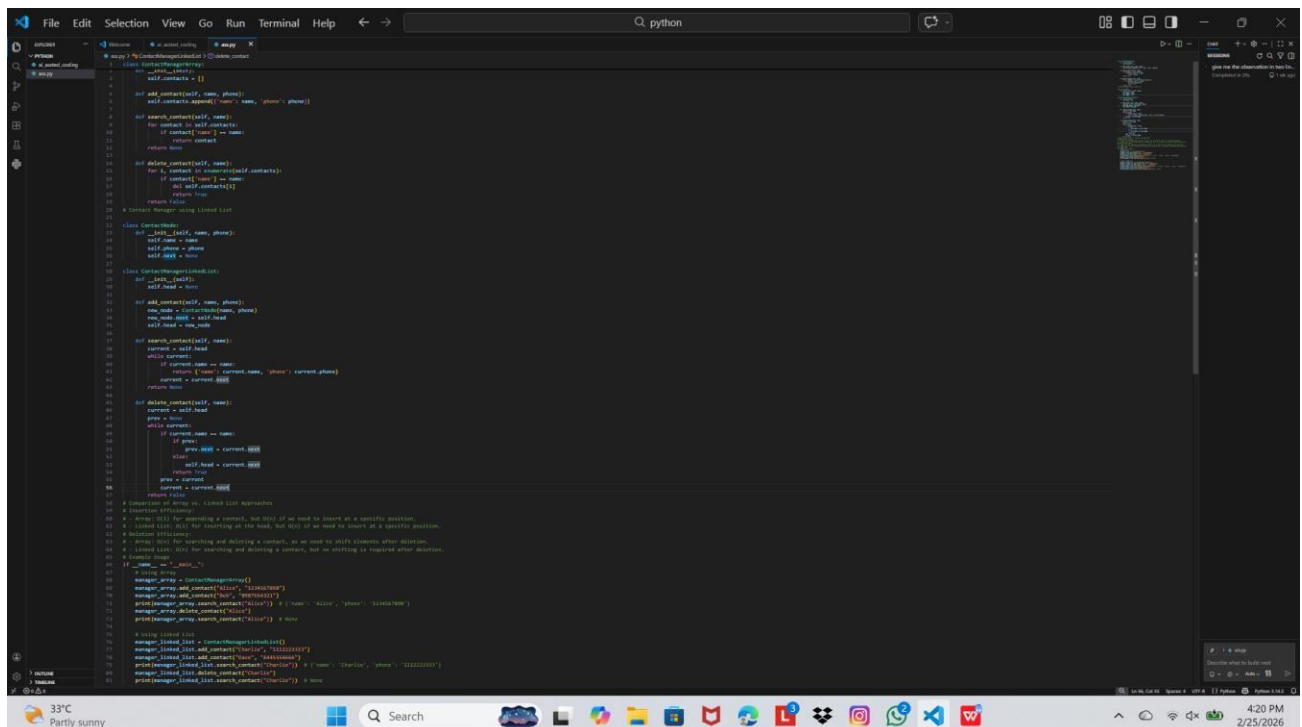
Batch: 20

Task 1: Smart Contact Manager (Arrays & Linked Lists)

Sample input

1. Implement the contact manager using arrays (lists).
2. Implement the same functionality using a linked list for dynamic memory allocation.
3. Implement the following operations in both approaches:
 - o Add a contact
 - o Search for a contact
 - o Delete a contact
4. Use GitHub Copilot to assist in generating search and delete methods.
5. Compare array vs. linked list approaches with respect to:
 - o Insertion efficiency
 - o Deletion efficiency

Code & probmt



```
File Edit Selection View Go Run Terminal Help python
# array
class ContactManagerArray:
    def __init__(self, name, phone):
        self.contacts.append({'name': name, 'phone': phone})

    def add_contact(self, name, phone):
        self.contacts.append({'name': name, 'phone': phone})

    def search_contact(self, name):
        for contact in self.contacts:
            if contact['name'] == name:
                return contact
        return None

    def delete_contact(self, name):
        for i, contact in enumerate(self.contacts):
            if contact['name'] == name:
                del self.contacts[i]
                return True
        return False

# linked list
class ContactManagerLinkedList:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
        self.next = None

    def add_contact(self, name, phone):
        new_node = ContactManagerLinkedList(name, phone)
        new_node.next = self.head
        self.head = new_node

    def search_contact(self, name):
        current = self.head
        while current:
            if current.name == name:
                return {'name': current.name, 'phone': current.phone}
            current = current.next
        return None

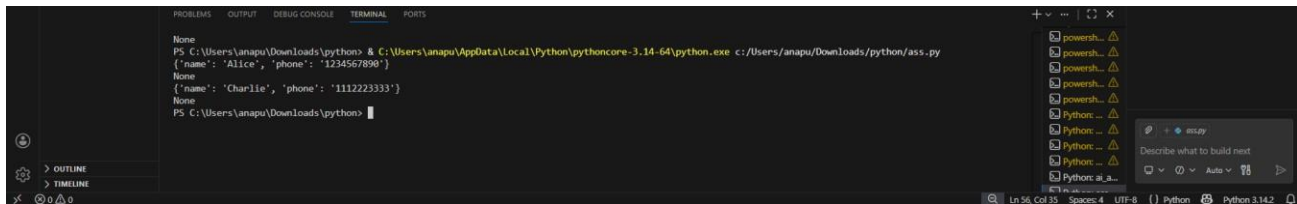
    def delete_contact(self, name):
        current = self.head
        prev = None
        while current:
            if current.name == name:
                if prev:
                    prev.next = current.next
                else:
                    self.head = current.next
                return True
            prev = current
            current = current.next
        return False

# Comparison of Array vs. Linked List Approaches
# Insertion Efficiency
# Array: O(1) for appending a contact, but O(N) if we need to insert at a specific position.
# Linked List: O(1) for inserting at the head, but O(N) if we need to insert at a specific position.
# Deletion Efficiency
# Array: O(N) for searching and deleting a contact, as we need to shift elements after deletion.
# Linked List: O(N) for searching and deleting a contact, but no shifting is required after deletion.

# Using Array
manager_array = ContactManagerArray()
manager_array.add_contact('Alice', '123456789')
manager_array.add_contact('Bob', '987654321')
print(manager_array.search_contact('Alice')) # {'name': 'Alice', 'phone': '123456789'}
manager_array.delete_contact('Alice')
print(manager_array.search_contact('Alice')) # None

# Using Linked List
manager_linked_list = ContactManagerLinkedList()
manager_linked_list.add_contact('Charlie', '1122334455')
manager_linked_list.add_contact('David', '5566778899')
print(manager_linked_list.search_contact('Charlie')) # {'name': 'Charlie', 'phone': '1122334455'}
manager_linked_list.delete_contact('Charlie')
print(manager_linked_list.search_contact('Charlie')) # None
```

Result:



```
None
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ass.py
{'name': 'Alice', 'phone': '1234567890'}
None
{'name': 'Charlie', 'phone': '1112223333'}
None
PS C:\Users\anapu\Downloads\python>
```

Observation:

- The array-based contact manager allows simple and fast appending ($O(1)$) but requires shifting elements during deletion, making removals $O(n)$.
- The linked list version enables efficient insertion at the head ($O(1)$) without shifting on deletion, though searching remains $O(n)$ in both approaches.

Task 2: Library Book Search System (Queues & Priority Queues)

Sample Input:

The SRU Library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests.

Tasks

1. Implement a Queue (FIFO) to manage book requests.
2. Extend the system to a Priority Queue, prioritizing faculty requests.
3. Use GitHub Copilot to assist in generating:
 - o enqueue() method
 - o dequeue() method
4. Test the system with a mix of student and faculty requests.

Prompt & Code:

```
Welcome  ai_assisted_coding  ass.py  X
ass.py > ...
1  class BookRequest:
2      def __init__(self, requester_type, book_title):
3          self.requester_type = requester_type # 'student' or 'faculty'
4          self.book_title = book_title
5  class Queue:
6      def __init__(self):
7          self.items = []
8
9      def enqueue(self, item):
10         self.items.append(item)
11
12         def dequeue(self):
13             if not self.is_empty():
14                 return self.items.pop(0)
15             return None
16
17         def is_empty(self):
18             return len(self.items) == 0
19  class PriorityQueue:
20      def __init__(self):
21          self.faculty_queue = Queue()
22          self.student_queue = Queue()
23
24          def enqueue(self, request):
25              if request.requester_type == 'faculty':
26                  self.faculty_queue.enqueue(request)
27              else:
28                  self.student_queue.enqueue(request)
29
30              def dequeue(self):
31                  if not self.faculty_queue.is_empty():
32                      return self.faculty_queue.dequeue()
33                  elif not self.student_queue.is_empty():
34                      return self.student_queue.dequeue()
35                  return None
36
37      # Example Usage
38      if __name__ == "__main__":
39          priority_queue = PriorityQueue()
40          priority_queue.enqueue(BookRequest('student', 'Book A'))
41          priority_queue.enqueue(BookRequest('faculty', 'Book B'))
42          priority_queue.enqueue(BookRequest('student', 'Book C'))
43
44          print(priority_queue.dequeue().book_title) # Book B (faculty request)
45          print(priority_queue.dequeue().book_title) # Book A (student request)
46          print(priority_queue.dequeue().book_title) # Book C (student request)
47          print(priority_queue.dequeue()) # None (no more requests)
```

Result:

```
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ass.py
Book B
Book A
Book C
None
PS C:\Users\anapu\Downloads\python>
```

Observation:

- The priority queue ensures faculty requests are always processed before student requests, regardless of arrival order.

- Within each category (faculty or student), requests are handled in FIFO order, preserving fairness among the same requester type

•

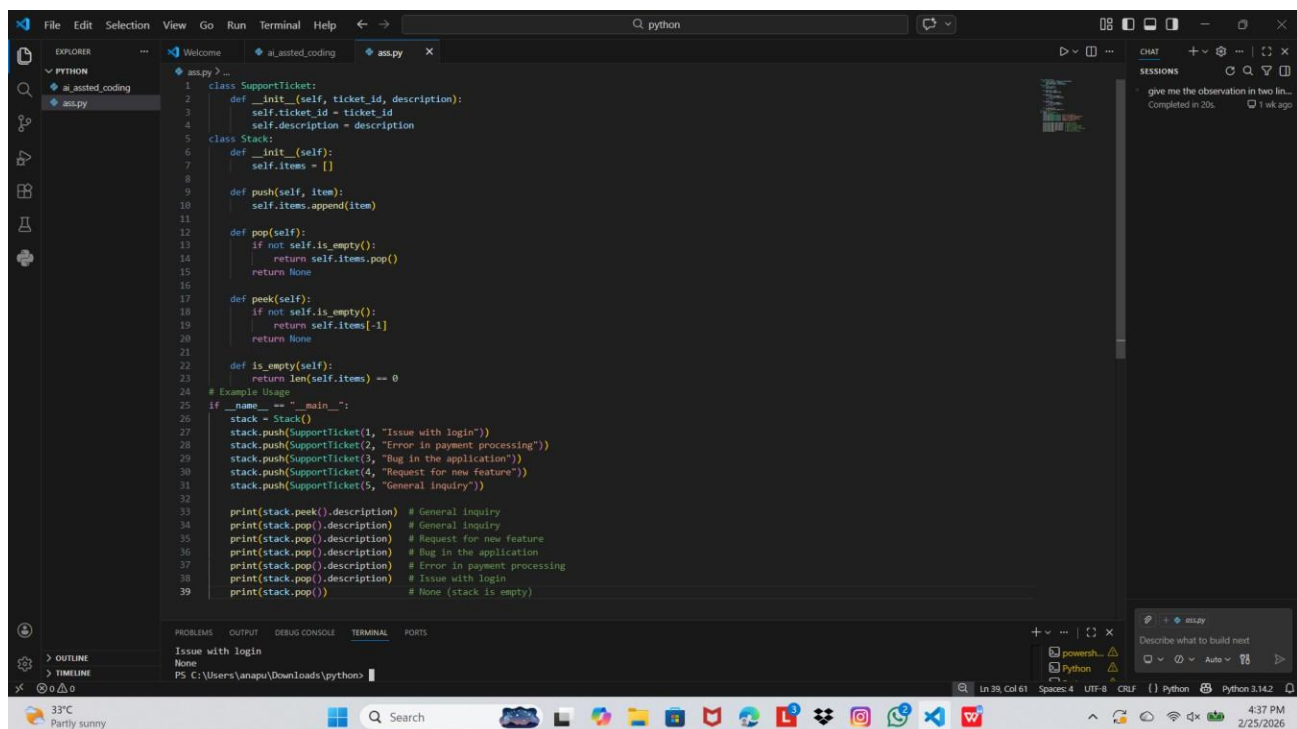
Task 3: Emergency Help Desk (Stack Implementation)

SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach.

Tasks

1. Implement a Stack to manage support tickets.
2. Provide the following operations:
 - o push(ticket)
 - o pop()
 - o peek()
3. Simulate at least five tickets being raised and resolved.
4. Use GitHub Copilot to suggest additional stack operations such as:
 - o Checking whether the stack is empty
 - o Checking whether the stack is full (if applicable)

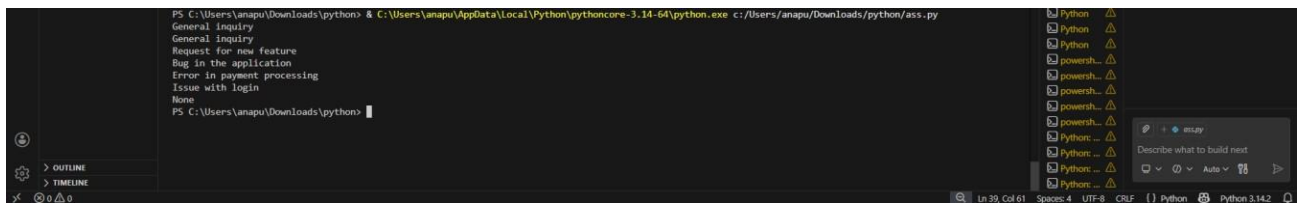
Prompt & Code:



```

1 class SupportTicket:
2     def __init__(self, ticket_id, description):
3         self.ticket_id = ticket_id
4         self.description = description
5
6 class Stack:
7     def __init__(self):
8         self.items = []
9
10    def push(self, item):
11        self.items.append(item)
12
13    def pop(self):
14        if not self.is_empty():
15            return self.items.pop()
16        return None
17
18    def peek(self):
19        if not self.is_empty():
20            return self.items[-1]
21        return None
22
23    def is_empty(self):
24        return len(self.items) == 0
25
26 # Example Usage
27 if __name__ == "__main__":
28     stack = Stack()
29     stack.push(SupportTicket(1, "Issue with login"))
30     stack.push(SupportTicket(2, "Error in payment processing"))
31     stack.push(SupportTicket(3, "Bug in the application"))
32     stack.push(SupportTicket(4, "Request for new feature"))
33     stack.push(SupportTicket(5, "General inquiry"))
34
35     print(stack.peek().description) # General Inquiry
36     print(stack.pop().description) # General Inquiry
37     print(stack.pop().description) # Request for new feature
38     print(stack.pop().description) # Bug in the application
39     print(stack.pop().description) # Error in payment processing
40     print(stack.pop().description) # Issue with login
41     print(stack.pop().description) # None (stack is empty)
  
```

Result:



Observation:

The stack follows the LIFO (Last In, First Out) principle, so the most recently added support ticket is processed first.

The `peek()` method allows viewing the top ticket without removing it, and attempting to `pop()` from an empty stack safely returns `None`

Task 4: Hash Table

Objective

To implement a Hash Table and understand collision handling

Sample input:

Task Description

Use AI to generate a hash table with:

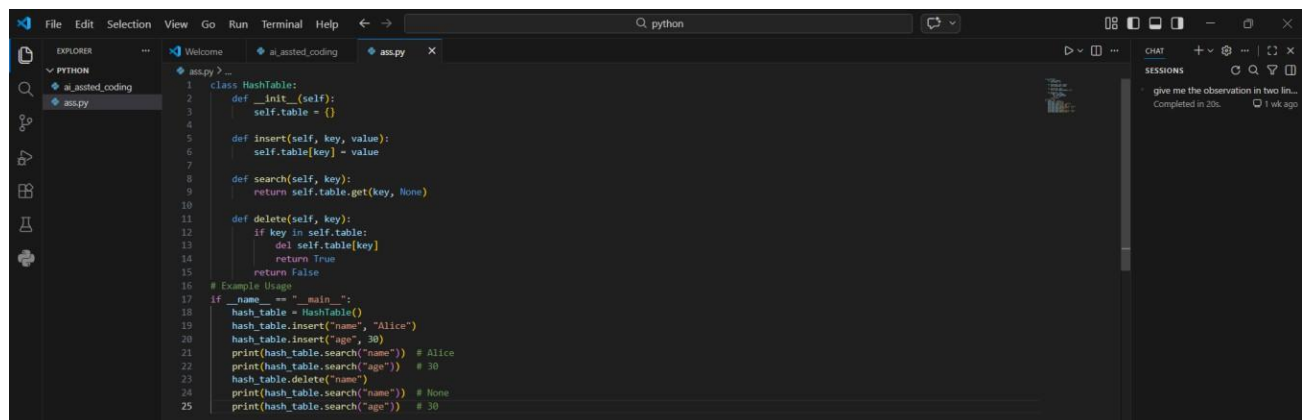
- Insert
- Search
- Delete

Starter Code

class HashTable:

pass

Prompt & Code:



```
1 class HashTable:
2     def __init__(self):
3         self.table = {}
4
5     def insert(self, key, value):
6         self.table[key] = value
7
8     def search(self, key):
9         return self.table.get(key, None)
10
11     def delete(self, key):
12         if key in self.table:
13             del self.table[key]
14             return True
15         return False
16
17 # Example Usage
18 if __name__ == "__main__":
19     hash_table = HashTable()
20     hash_table.insert("name", "Alice")
21     hash_table.insert("age", 30)
22     print(hash_table.search("name")) # Alice
23     print(hash_table.search("age")) # 30
24     hash_table.delete("name")
25     print(hash_table.search("name")) # None
26     print(hash_table.search("age")) # 30
```

Result:



```
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:\users\anapu\Downloads\python\ass.py
Alice
30
None
30
PS C:\Users\anapu\Downloads\python>
```

Observation

The hash table provides fast average-case time complexity of $O(1)$ for insertion, search, and deletion operations using key-based access.

After deleting a key, searching for it returns `None`, while other existing keys remain unaffected in the table.

Task 5: Real-Time Application Challenge

Sample input:

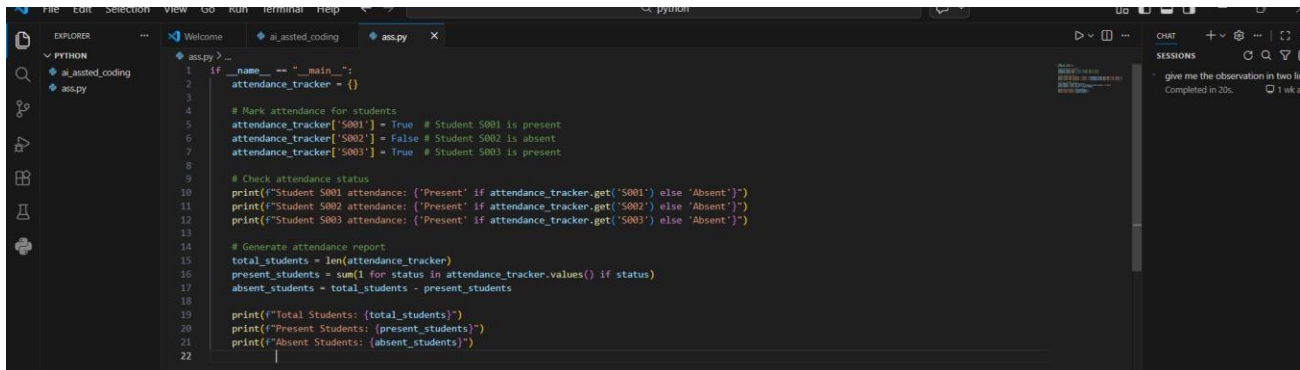
Design a Campus Resource Management System with the following features:

- Student Attendance Tracking
- Event Registration System
- Library Book Borrowing
- Bus Scheduling System
- Cafeteria Order Queue

Student Tasks

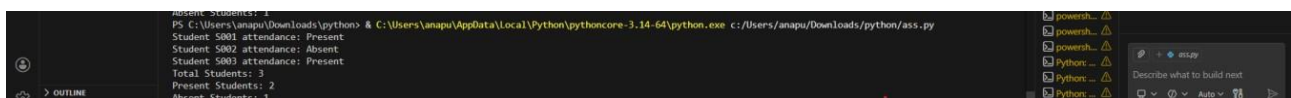
1. Choose the most appropriate data structure for each feature.
2. Justify your choice in 2–3 sentences.
3. Implement one selected feature using AI-assisted code generation.

Prompt & Code:



```
1 if __name__ == "__main__":
2     attendance_tracker = {}
3
4     # Mark attendance for students
5     attendance_tracker['S001'] = True # Student S001 is present
6     attendance_tracker['S002'] = False # Student S002 is absent
7     attendance_tracker['S003'] = True # Student S003 is present
8
9     # Check attendance status
10    print(f"Student S001 attendance: {'Present' if attendance_tracker.get('S001') else 'Absent'}")
11    print(f"Student S002 attendance: {'Present' if attendance_tracker.get('S002') else 'Absent'}")
12    print(f"Student S003 attendance: {'Present' if attendance_tracker.get('S003') else 'Absent'}")
13
14    # Generate attendance report
15    total_students = len(attendance_tracker)
16    present_students = sum(1 for status in attendance_tracker.values() if status)
17    absent_students = total_students - present_students
18
19    print(f"Total Students: {total_students}")
20    print(f"Present Students: {present_students}")
21    print(f"Absent Students: {absent_students}")
22
```

Result:



```
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ass.py
Absent Students: 1
Student S001 attendance: Present
Student S002 attendance: Absent
Student S003 attendance: Present
Total Students: 3
Present Students: 2
Absent Students: 1
```


Observation:

The dictionary efficiently tracks attendance using student IDs as keys, allowing quick $O(1)$ access to each student's status.

The attendance report is generated by counting boolean values, making it simple to calculate total, present, and absent students dynamically.